



Comenzado el	martes, 15 de septiembre de 2026, 22:14		
Estado	Finalizado		
Finalizado en	miércoles, 16 de septiembre de 2026, 01:29		
Tiempo empleado	3 horas 14 minutos		
Calificación	7,00 de 10,00 (70%)		

Pregunta 1

Incorrecta

Se puntúa 0,00 sobre 1,00

Recibimos el listado de tuplas película/año de estreno disponibles películas estrenadas, y queremos devolver un listado de películas agrupadas por año (es decir, en una tupla de la forma (Año, [Título])).

```
listaPorAnio :: [(String, Integer)] -> [(Integer, [String])]
```

```
listaPorAnio [] = []
```

```
listaPorAnio ((pelicula, anio):xs) = agregarPelicula pelicula anio (listaPorAnio xs)
```

Determinar cuál de las siguientes opciones es la correcta, según el problema indicado:

- a. `agregarPelicula : String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]`
`agregarPelicula pelicula anio [(a, peliculas)] xs | anio == a -> (a, pelicula : peliculas) : xs`
`| otherwise -> (a, peliculas) : agregarPelicula pelicula anio xs`

b. `agregarPelicula : String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]`
`agregarPelicula pelicula anio [(a, peliculas)] xs | anio == a -> (a, pelicula : peliculas) : xs`
`| otherwise -> agregarPelicula pelicula anio xs`

☒ c. `agregarPelicula : String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]` ✖ La respuesta correcta es
`agregarPelicula : String -> Integer -> [(Integer, [String])]`
`agregarPelicula pelicula anio [(a, peliculas)] xs | anio == a -> (a, pelicula : peliculas) : xs`
`| otherwise -> agregarPelicula pelicula anio xs`
 El caso base no puede devolver 'otherwise' es necesario agregar

☐ d. `agregarPelicula : String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]`
`agregarPelicula pelicula anio [(a, peliculas)] xs | anio == a -> (a, pelicula : peliculas) : xs`
`| otherwise -> agregarPelicula pelicula anio xs`


```

La respuesta correcta es
agregarPelicula :: String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]
agregarPelicula pelicula anio l = [(anio, [pelicula])]
agregarPelicula pelicula anio (a, peliculas:xs) | anio == a = (a, pelicula : peliculas) : xs
| otherwise = (a, peliculas) : agregarPelicula pelicula anio xs

```

El caso base no puede devolver la lista vacía, ya que lo que queremos es agregar una nueva película en caso de que la lista esté vacía; además, en el caso recursivo del "otherwise" es necesario agregar la tupla actual a la salida, porque queremos dejarla intacta sin modificarle ni el año ni la lista de películas.

Respuesta incorrecta.

La respuesta correcta es: `agregarPelicula :: String -> Integer -> [(Integer, [String])] -> [(Integer, [String])]`

```
agregarPelicula pelicula anio [] = [(anio, [pelicula])]
```

```
agregarPelicula pelicula anio ((a, peliculas):xs) | anio == a = (a, pelicula : peliculas) : xs
                        | otherwise = (a, peliculas) : agregarPelicula pelicula anio xs
```

Pregunta 2

Correcta

Se puntúa 1.00 sobre 1.00

Sea $m: \text{seq}(\text{seq}(Z))$ una entrada de un problema. El siguiente asegura:

```
asegura: (|resultado|-m)
```

asegura que resultado es una matriz de las mismas dimensiones que 'm'.

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es "Falso", ya que resultado no necesariamente tiene que ser una matriz. Por ejemplo, un vector que tenga tantos elementos como filas tiene "m" cumple con el asegura indicado.

La respuesta correcta es 'Falso'

Se puntúa 1,00
sobre 1,00

```
aproboMasDeNMaterias registro alumno n = longitud (fun2 (fun1 alumno registro)) >= n --longitud es una función auxiliar que devuelve la longitud de una lista
```

- ```
| otherwise = fun2 xs
```

- ```
| otherwise = fun2 xs
```

- ```
| otherwise = fun2 xs
```

- ```
fun2 (x:xs) | x >= 4 =
```

- sta correcta

Vemos que fun1 no necesita un llamado recursivo en el caso 'a == b', ya que la especificación aclara que no hay nombres repetidos en el registro. Por otro lado, fun2 si necesita el caso recursivo para 'x>=4', ya que queremos devolver *todas* las notas aprobadas, no sólo la primera.

```
| otherwise = fun2 xs
```

Se puntúa 1,00 sobre 1,00

```
letraANatural a = (ord a) - (ord 'a')
```

```
| otherwise = x : (cifrar xs n)
```

- f. `desplazar :: Char -> Int -> Char`
`desplazar a n | esMinuscula a = naturalALetra ((letraANatural a) + n)`
`| otherwise = a`

- La respuesta correcta es: desplazar

| otherwise = a

donde la operación "mod" hace que sea cíclica la operación, permitiendo que sigamos teniendo caracteres por arriba de 'z' y por abajo de 'a'. Por otra parte, `naturalALetter` y `letterANatural` tienen tipos específicos que deben respetar su entrada y salida, por lo que el orden indicado es el correcto.

Pregunta 5

Incorrecta

Se puntúa 0.00 sobre 1.00

Sea la siguiente implementación de la función "multiplicarMatrizPorUnEscalar", donde "A" y "B" reemplazan dos operaciones de listas:

```
multiplicarMatrizPorEscalar :: Integer -> [[Integer]] -> [[Integer]]
multiplicarMatrizPorEscalar _ [] = []
multiplicarMatrizPorEscalar c (x:xs) = multiplicarFila c x A multiplicarMatrizPorEscalar c xs

multiplicarFila :: Integer -> [Integer] -> [Integer]
multiplicarFila c [x] = [c*x]
multiplicarFila c (x:xs) = c*x B multiplicarFila c xs
```

Para que la implementación sea correcta, debe sustituirse:

- ☒ a. 'A' por ":", 'B' por "++" ✗ Respuesta correcta: ambas deben sustituirse por ":", ya que en ambos casos estoy añadiendo un elemento al comienzo de una lista.
- ☐ b. 'A' por ":", 'B' por ":"
- ☐ c. 'A' por "++", 'B' por "++"
- ☐ d. 'A' por "++", 'B' por ":"

Respuesta incorrecta.

La respuesta correcta es: "A" por ":", "B" por ":"

Pregunta 6

Correcta

Se puntúa 1.00 sobre 1.00

Sea la siguiente implementación (con un error) del problema de encontrar la ubicación de un locker dentro de un mapa de lockers:

```
type Texto = [Char]
type Identificacion = Integer
type Ubicacion = Texto
type Estado = (Disponibilidad, Ubicacion)
type Locker = (Identificacion, Estado)
type MapaDeLockers = [Locker]
type Disponibilidad = Bool

ubicacionDelLocker :: Identificacion -> MapaDeLockers -> Ubicacion
ubicacionDelLocker _ [] = "No está el locker en el mapa"
ubicacionDelLocker n (x:xs) | n == fst x = snd x
                             | otherwise = ubicacionDelLocker n xs
```

El error de la implementación es:

- ☐ a. El guión bajo "_" debería ser reemplazado por una variable
- ☐ b. "fst x" no devuelve algo de tipo comparable con "n"
- ☒ c. "snd x" no es la salida correcta ✓ Respuesta correcta: "snd x" debe ser modificado a "snd (snd x)", ya que queremos acceder al segundo elemento del segundo elemento de "x" (dos tuplas anidadas). No hay que agregar "x:" o "++ [x]" para el problema
- ☐ d. En el llamado recursivo, falta agregar "x:", de forma que el llamado recursivo sea "x : ubicacionDelLocker n xs"
- ☐ e. En el llamado recursivo, falta agregar " ++ [x]", de forma que el llamado recursivo sea "ubicacionDelLocker n xs ++ [x]"

Respuesta correcta

La respuesta correcta es: "snd x" no es la salida correcta para el problema

Pregunta 7

Correcta

Se puntúa 1.00 sobre 1.00

Recibimos un listado con todos los libros disponibles en una biblioteca. Cada libro tiene un autor y un título (primera y segunda componente del par respectivamente). El objetivo es crear un catálogo, es decir una lista de tuplas cuya primera componente es el autor y la segunda es la lista de todos sus títulos: problema "catalogoPorAutores", donde res es un catálogo donde cada autor aparece una sola vez junto con la lista de todos sus títulos.

```
catalogoPorAutores :: [(String, String)] -> [(String, [String])]
catalogoPorAutores [] = []
catalogoPorAutores (autor, titulo):xs | estaEnCatalogo catalogoXs autor = agregarTitulo catalogoXs autor titulo
                                     | otherwise = (autor, [titulo]) : catalogoXs
                                     where catalogoXs = catalogoPorAutores xs

estaEnCatalogo :: [(String, [String])] -> String -> Bool
estaEnCatalogo [] autor = False
estaEnCatalogo (c:cs) autor = fst c == autor || estaEnCatalogo cs autor
```

Determinar cuál de las siguientes opciones es la correcta, según el problema indicado:

- ☐ a. agregarTitulo :: [(String, [String])] -> String -> String -> [(String, [String])]
agregarTitulo [] autor titulo = []
agregarTitulo (c:cs) autor titulo | (head c) == autor = (autor, titulo : (tail c)) : (agregarTitulo cs autor titulo)
| otherwise = c : (agregarTitulo cs autor titulo)
- ☒ b. agregarTitulo :: [(String, [String])] -> String -> String -> [(String, [String])] ✓ La respuesta correcta es
agregarTitulo :: [(String, [String])] -> String -> String -> [(String, [String])]
agregarTitulo [] autor titulo = []
agregarTitulo (c:cs) autor titulo | fst c == autor = (autor, titulo : (snd c)) : (agregarTitulo cs autor titulo)
| otherwise = c : (agregarTitulo cs autor titulo)

Como c es una tupla, debo acceder a ambos valores mediante las funciones fst y snd respectivamente. Las funciones de head y tail funcionan con tipos de entrada de listas, por lo que no es correcto usarlas en tuplas.
- ☐ c. agregarTitulo :: [(String, [String])] -> String -> String -> [(String, [String])]
agregarTitulo [] autor titulo = []
agregarTitulo (c:cs) autor titulo | c == autor = (autor, titulo) : (agregarTitulo cs autor titulo)
| otherwise = c : (agregarTitulo cs autor titulo)

Respuesta correcta

La respuesta correcta es: agregarTitulo :: [(String, [String])] -> String -> String -> [(String, [String])]
agregarTitulo [] autor titulo = []
agregarTitulo (c:cs) autor titulo | fst c == autor = (autor, titulo : (snd c)) : (agregarTitulo cs autor titulo)
| otherwise = c : (agregarTitulo cs autor titulo)

Pregunta 8

Correcta

Se puntúa 1.00 sobre 1.00

Sea m: seq(seq(Z)) una entrada de un problema. Los siguientes requiere:

requiere: { |m| > 0 }
requiere: { |m[0]| > 0 }

- ☐ a. "m" es una matriz con todos sus elementos mayores a cero
- ☐ b. "m" es una matriz cuadrada
- ☐ c. "m" puede ser una lista vacía
- ☐ d. Todas las anteriores
- ☒ e. Ninguna de las anteriores ✓ Respuesta correcta: Ninguna de las anteriores. Los requiere establecen que "m" tenga al menos una fila y una columna, pero no necesariamente la misma cantidad de ellas, por lo que no necesariamente es una matriz cuadrada.

Respuesta correcta

La respuesta correcta es: Ninguna de las anteriores

Pregunta 9

Incorrecta

Se puntúa 0.00 sobre 1.00

Sea la siguiente implementación de la función "concatenarFilas", donde "A" reemplaza una operación de listas:

```
concatenarFilas :: [[String]] -> [String]
concatenarFilas [c] = c
concatenarFilas (x:xs) = x A concatenarFilas xs
```

Para que la implementación sea correcta, debe sustituirse:

- ☒ a. 'A' por ":" ✗ Respuesta correcta: Debe sustituirse por "++", ya que se están uniendo dos listas.
- ☐ b. 'A' por "++"

Respuesta incorrecta.

La respuesta correcta es: "A" por "++"

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

La siguiente implementación de la función 'quitar', con entradas como un elemento 'a' y una lista 's', elimina la primera aparición de 'a' de la lista 's'.

quitar :: (Eq t) => t -> [t] -> [t]

quitar a [] = []

quitar a (x:xs) | a == x = xs

 | otherwise = x : quitar a xs

¿Cómo extendería la función para que elimine todas las instancias de 'a' de la lista 's', modificando o agregando una única línea?

- ☒ a. Agregaría un llamado recursivo adicional en la línea 2 ('quitar a (x:xs) | a == x = xs')
- ☐ b. Ninguna de las otras opciones
- ☐ c. Modificaría el llamado recursivo existente en la línea 3 ('otherwise = x : quitar a xs')
- ☐ d. Agregaría otro caso base



Modificando una sola línea, es necesario agregar un llamado recursivo adicional para el caso positivo, es decir, cuando 'a == x', para asegurarnos que pese a haber encontrado una instancia de 'a' debe seguir buscando en el resto de la fila. Quedaría entonces: 'a == x = quitar a xs'.

Respuesta correcta

La respuesta correcta es: Agregaría un llamado recursivo adicional en la línea 2 ('quitar a (x:xs) | a == x = xs')



Actividad previa

14 de septiembre, labo 1105

Ir a...



Siguiente actividad

[Avisos](#)

